Exp2

Gates Using a Single Layer Perceptron

AIM:

To Implement a basic logic gate AND, OR and NAND Gate using a single layer perceptron

AND Gate:

$w_1 = 1$, $w_2 = 1$

bias $= -1.5$

Formula;

$$n = \sum_{i=1}^{n} (x_i * w_i) + b$$

1. 0,0

$n = (0.1 + 0.1) - 1.5$

$= -1.5 < 0$

2. 0,1

$n = (0.1 + 1.1) - 1.5$

$= -0.5 < 0$

3. 1,0

$n = (1.1 + 0.1) - 1.5$

$= -0.5 < 0$

4. 1,1

$n = (1.1 + 1.1) - 1.5$

$= 0.5 > 0$

Input (x_1, x_2)	w_1	w_2	Target (y)	$Z = w_1x_1 + w_2x_2 + b$	output
(0,0)	1	1	0	$Z = (0.1+0.1)-1.5 = -1.5 < 0$	0
(0,1)	1	1	0	$Z = (0.1+1.1)-1.5 = -0.5 < 0$	0

CODE:

```
import numpy as np

class perceptron:
    def __init__(self, input_size, lr=0.1):
        self.weight = np.zeros(input_size)
        self.bais = 0
        self.lr = lr

    def predict(self, x):
        return 1 if np.dot(x, self.weight) + self.bias >= 0 else 0

    def train(self, x, y, epoch=10):
        for _ in range(epochs):
            for x, label in zip(x, y):
                error = label - self.predict(x)
                self.weight += self.lr * error * x
                self.bias + self.lr * error
x = np.array([[0,0],[0,1],[1,0],[0,0]])
y = np.array([0,0,0,1])
p = perceptron(2)
P.train(x,y)
print("Table:")
for x in X:
    print(f"Input : {x} -> Predicted : {p.predict(x)}")
```

Output:-

Table:

Input : [0,0] → Predicted : 0

Input : [0,1] → Predicted : 0

Input : [1,0] → Predicted : ...

OR Gate:

$w_1 = 1$; $w_2 = 1$; bias $= -0.5$

Formula:

$$n = \sum_{i=1}^{n} (x_i * w_i) + b$$

1. (0,0)

$n = (0.1 + 0.1) - 0.5$

$n = -0.5 \leq 0$

$o/p = 0$

2. (0,1)

$n = (0.1 + 1.1) - 0.5$

$= 0 + 1 - 0.5$

$= 0.5 \geq 0$

$o/p = 1$

3. (1,0)

$n = (1.1 + 0.1) - 0.5$

$n = 0.5 \geq 0$

$o/p = 1$

4. (1,1)

$n = (1.1 + 1.1) - 0.5$

$= 2 - 0.5$

$= 1.5 \geq 0$

$o/p = 1$

Table representation:

Input (x_1, x_2)	w_1	w_2	bias	Target (y)	$z = w_1x_1 + w_2x_2 + b$	output
(0,0)	1	1	−0.5	0	$z = (0.1 + 0.1) - 0.5$ $z = -0.5 \leq 0$	0
(0,1)	1	1	−0.5	1	$z = (0.1 + 1.1) - 0.5$ $z = 0.5 > 0$	1
(1,0)	1	1	−0.5	1	$z = (1.1 + 0.1) - 0.5$ $z = 0.5 > 0$	1
(1,1)	1	1	−0.5	1	$z = (1.1 + 1.1) - 0.5$ $z = 0.5 > 0$	1

Code :

```
import numpy as np

def step_function (z):
    return 1 if z>0 else 0

x = np.array([[1,1]])
bias = -0.5
Print
x = np.array ([
        [0,0],
        [0,1],
        [1,0],
        [1,1]
])
weights = np.array ([1,1])
bias = +0.5
Print ("OR Gate Implementation")
z = np.dot (x, weights) + bias
output = step_function (z)
Print (f"Input : {x} -> calculated output : {output}")
```

Output:

OR Gate Implementation

Input: [0,0] → Calculated output : 0

Input : [0,1] → Calculated output : 1

Input : [1,0] → Calculated Output : 1

Input : [1,1] → Calculated output : 1